Feedback and decisions: where this design came from

22 October 2026 · COMP1110 Group 08, Empty Classroom Plan · Square brackets [ ] are for the group to fill from their own notes

1. How the design was decided

Two sources decided this design.

First, a group discussion. We started from the Plan, which says the University timetable is an input we read rather than something we build. We asked what an outside service would have to provide for our system to work, and what we would need if we never got access to the real one.

Second, feedback from friends outside the group. We described the idea to a few people who are not on the team, and asked three questions: does this make sense, what would you worry about, and what would you want to see. Their comments are in the first table.

2. What our friends raised, and what we changed

#	Who	What they raised (in our words)	What we changed
1	[friend, course]	[ ]	[ ]
2	[friend, course]	[ ]	[ ]
3	[friend, course]	[ ]	[ ]
4	[friend, course]	[ ]	[ ]

3. Decisions, and who they came from

Tick one source per row, then keep this table as an appendix.

Decision	Why we made it	Source
The feed is a separate service with its own database, not a shared schema	Only a real boundary can be tested: network failure, slow responses, stale data	[group / friends]
The two systems share only the API contract	Either side can change without breaking the other; no coupling through code or files	[group / friends]
The service is read-only for us, with a separate write side for the registrar	We must not disturb the University's system, and we should not be able to	[group / friends]
Polling the metadata, not the timetable	The timetable changes a few times a day, not every second; a cheap GET /v1/meta tells us if anything changed	[group / friends]
lastUpdated and revision exist as first-class fields	Without them the consumer cannot tell fresh data from stale, and cannot detect change cheaply	[group / friends]
Overlap rejection lives in the feed, not in our system	That rule belongs to the source of the timetable; enforcing it ourselves would pretend we own the data	[group / friends]
sockets is null when unknown, never false	We do not know and there are no sockets are different facts, and a student acts differently on each	[group / friends]
Test switches for stale, slow and down	The behaviour we most need to test is failure behaviour, and it cannot be produced on demand otherwise	[group / friends]
A browser control panel served by the same service	We needed to see the timetable as a grid and change data quickly while developing	[group / friends]

4. What we discussed and deliberately did not do

• Pushing instead of polling. A university registry will not add a webhook for a student project. [confirm]

• Connecting to a University database directly. It would be intrusive and we have no permission. [confirm]

• Enforcing timetable rules in our own system. That would duplicate the source of truth. [confirm]

• A real login system for the feed. One fixed API key is enough to test with and without permission. [confirm]

5. What we know we cannot detect

A change in the meaning of a field cannot be caught automatically. If classes moved from ending at :50 to ending at :55, the data would still parse and our answer would be quietly wrong. Our answer is a diff against the previous revision with a threshold, plus a person looking at it. We wrote this down as a limitation instead of claiming the system handles it.

6. Who did what

Person	Contribution to this part
[name]	[ ]
[name]	[ ]
[name]	[ ]
[name]	[ ]
[name]	[ ]

Appendix: the architecture these decisions produce

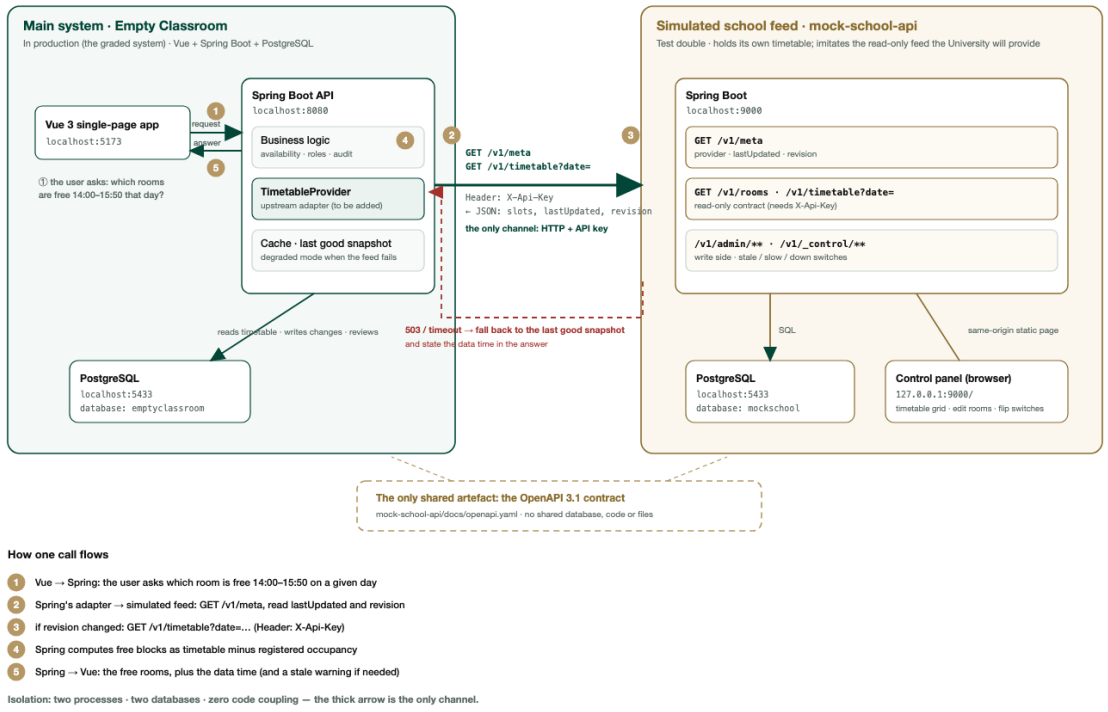


Figure 1. The main system and the simulated feed. Numbers 1 to 5 mark the call path.

The architecture these decisions produce is in figures/two-systems-architecture-en.png. The reasoning is in design-log-2026-10-22.md.